

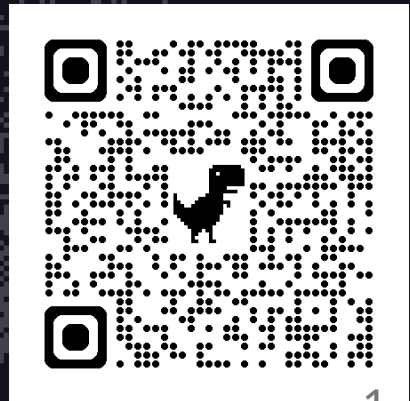
Every Map an Evolution, Every Room a Generation: Co-Evolution in a Procedurally-Generated Video Game

Erik Fredericks, frederer@gvsu.edu

Winter 2026

<https://efredericks.github.io>

(Background image: Etienne Jacob)



Premise

Procedural Content Generation (PCG)

What is PCG?

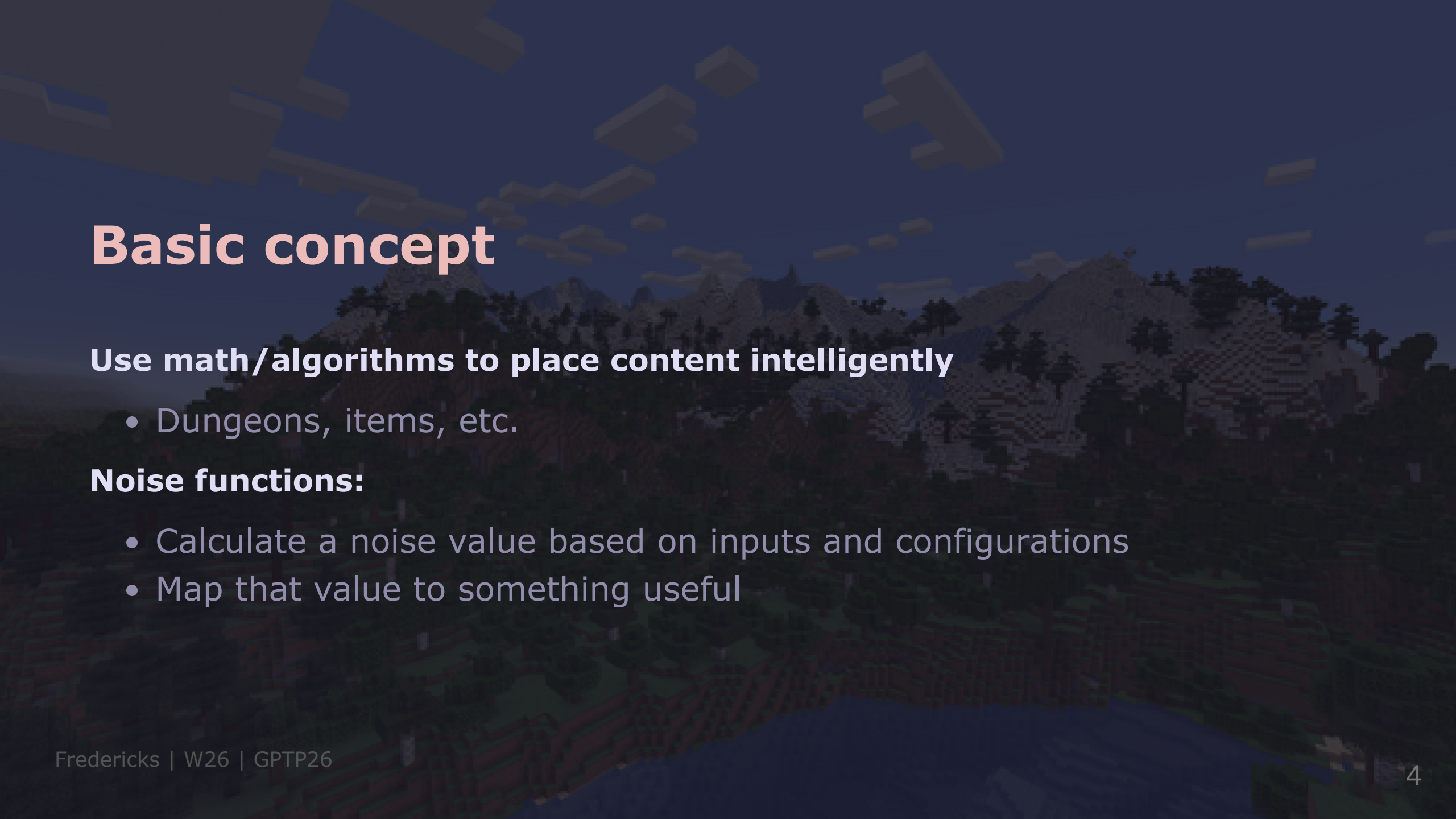
- Algorithmically-placing and generating content

Why PCG?

- Save the developer/designer time and effort!
 - One would think...

Goal for GP?

- Optimize for variety and difficulty in game experience



Basic concept

Use math/algorithms to place content intelligently

- Dungeons, items, etc.

Noise functions:

- Calculate a noise value based on inputs and configurations
- Map that value to something useful

For example - Perlin noise []

Typically, returns a value in [-1.0, 1.0] (p5js uses [0.0, 1.0])

For example (in p5):

```
let n = noise(x * 0.01, y * 0.01);
```

- `n = [0.0, 0.5]` -- (x, y) -> water
- `n = (0.5, 0.6]` -- (x, y) -> beach
- `n = (0.6, 0.9]` -- (x, y) -> grass
- `n = (0.9, 1.0]` -- (x, y) -> rock

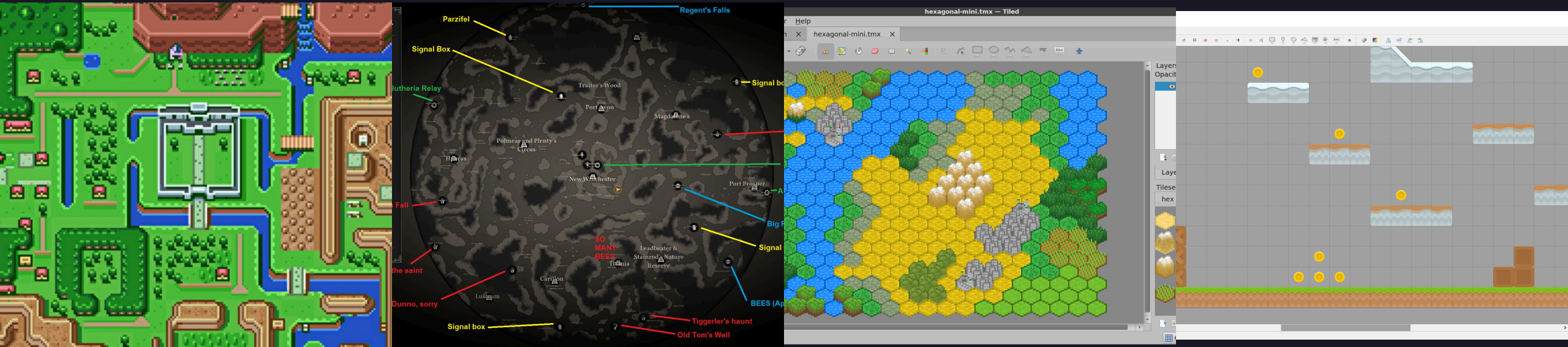
There are other noise functions! Worley noise, Simplex noise, etc.

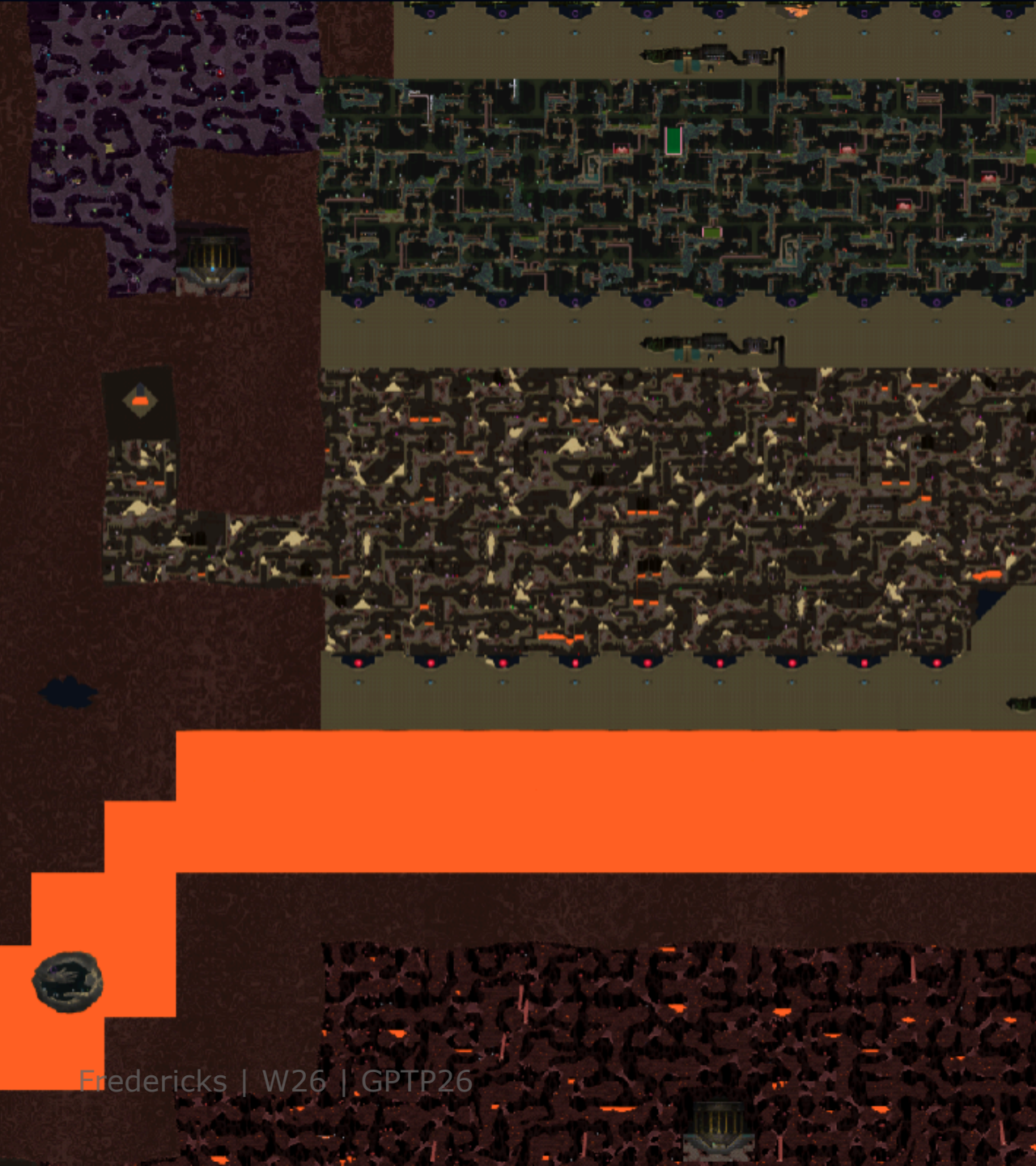


Noise function mappings:

- Environment biome
- Map height
- Enemy spawns

Making a game map





Fredericks | W26 | GPTP26



x1?

Time: 00:32:32

Fitness Functions

Next Steps

References

Demo time

This time I made a pre-baked p5js template for you. Go to this link and

File -> Duplicate

- Make sure you login and save often

The sketch

- You should have a little ASCII happy face that you can move around with your arrow keys
- Your game map is represented as a **two-dimensional grid** where the first index is the row and the second index is the column
- There is also a basic camera that follows your player so that we can make big maps

The map

```
[['#', '#', '#', ..., '#'],  
 ['#', ' ', ' ', ..., '#'],  
 ['#', '.', 't', ..., '#'],  
 [...],  
 ['#', '#', '#', ..., '#']]
```

`game_map[2][1]` returns a ???

- `.`

By default, the map puts walls on the outside and nothing on the inside - your job is to fill it with things

Try hitting ~ and see what happens!

Valid things:

Character	Walkable?	Represents
#	No	Stone wall
t	No	Tree
w	No	Water
	Yes	Empty space
.	Yes	Pebbles
g	Yes	Grass



Try setting some random cells to these characters!

- such as, `game_map[2][3] = 'g';` in `setup()` AFTER the game map is created

First, let's randomize things

After the call to `setupGameMap()` (which again, just gives us an empty grid with borders)

```
for (let r = 1; r < num_rows - 1; r++) {  
  for (let c = 1; c < num_cols - 1; c++) {  
    let r = random();  
    let ch = ' ';  
    if (r > 0.8) ch = '#';  
  
    game_map[r][c] = ch;  
  }  
}
```

Now let's give it a bit of detail

First, comment out what you did inside that double loop.

```
const zoom = 0.01;
for (let r = 1; r < num_rows - 1; r++) {
  for (let c = 1; c < num_cols - 1; c++) {
    let ch = '';
    let n = noise(c * zoom, r * zoom);

    // fill in with code from next slide

    game_map[r][c] = ch;
  }
}
```

The noise bit

```
if (n <= 0.5) ch = 'w';  
else if (n <= 0.6) ch = 'b';  
else if (n <= 0.9) ch = 't';  
else {  
    let r = random();  
    if (r > 0.8) ch = '.';  
    else ch = ' ';  
}
```

The finesse bit

Now this is the tricky part - you need to play with the `zoom` value and the `noiseDetail` (at the top) values to get exactly the output you want!

Try varying the:

- `zoom` into the noise distribution (try `0.1`, `0.001`, etc.)
- <https://p5js.org/reference/p5/noiseDetail/>
 - The number of octaves (first parameter in `noiseDetail`) - `[1, 16]` usually show interesting values
 - The falloff amount (second parameter in `noiseDetail`) - `[0.0, 1.0]`

Other ways to do it!

Use a different algorithm!

- Rectangular room placement
- Cellular automata
- Wave function collapse
- ...

